

CHAPTER 1

INTRODUCTION

Examinations are one of the most important ways to evaluate students' knowledge and abilities, serving as a foundation for academic growth and achievement. However, maintaining fairness during exams can be a significant challenge, as dishonest practices like cheating and unauthorized activities continue to be a persistent problem. These actions not only disrupt the examination process but also undermine the credibility of academic systems. To address these concerns, this project introduces an Anomaly Detection System for Exam Halls designed to automatically identify and flag suspicious activities.

Cheating in exams can take many forms, including passing chits, writing answers on body parts like hands, peeking at another student's paper, discussing answers, showing answers intentionally, using mobile phones, or referring to hidden notes. Detecting such activities manually is not only difficult but also prone to human error. Invigilators can become fatigued during long exams, and subtle cheating tactics can easily go unnoticed. Similarly, while CCTV systems provide constant monitoring, analyzing the footage can be time-consuming and resource-intensive. Additionally, these methods often raise privacy concerns for students.

This project offers a modern, technology-driven solution by using video processing and machine learning to detect these specific cheating activities. Instead of relying on live monitoring or manual footage review, the system analyzes video by breaking it into individual frames, allowing for detailed observation of each moment. It uses a labeled dataset to train a machine learning model that can accurately distinguish between normal and suspicious activities, such as looking at answers written on a hand or passing a note to another student.

Unlike invigilators, it doesn't get tired, and it applies consistent standards across all students. By automating the process, the system reduces the workload on invigilators while improving accuracy in detecting irregularities. It's also designed to be scalable, meaning it can adapt to different exam setups and handle large numbers of students without extra effort. Privacy is another critical advantage; the system focuses on individual frames rather than continuous video streams, and the data is processed in a way that ensures ethical compliance.

1.1 Area of Domain

This project falls under the core domain of Machine Learning (ML), with its sub-domains being Image Processing and Pattern Recognition, integral components of computer vision. Machine Learning involves developing algorithms that allow systems to learn patterns and make predictions or decisions without being explicitly programmed for specific tasks. It enables intelligent systems to adapt and improve as they are exposed to more data over time.

Image Processing: This involves preprocessing visual data to enhance its quality and usability for analysis. Techniques like resizing, normalization, grayscale conversion, and data augmentation are used to ensure the data is optimized for machine learning models.

Pattern Recognition: This focuses on identifying and classifying patterns within data, particularly visual data in this project. By leveraging trained Machine learning models, pattern recognition enables the detection of anomalies and classification of activities, such as "Using Mobile" or "Copying," by analyzing spatial and temporal patterns in the data. It is a key component in applications like anomaly detection, activity analysis, and activity recognition.

1.2 Overview of the Project

This project focuses on developing a multi-class anomaly detection system tailored for offline exam monitoring, aiming to identify and categorize activities such as copying, chit passing, discussing, and using mobile devices. Utilizing a convolutional neural network (CNN) trained on a labeled dataset of suspicious activities images, the system processes video footage by extracting frames, resizing them, and applying normalization to enhance model performance. The model is capable of classifying frames into one of nine predefined classes with high accuracy, leveraging advanced data preprocessing techniques like augmentation, grayscale conversion, and feature extraction. Integrated with Streamlit for an interactive user interface, the system offers real-time anomaly detection and visualization.

To effectively detect these suspicious activity , the project follows a structured workflow:

- **Preprocessing Exam Hall Videos**

Videos from the exam hall are divided into individual frames, allowing the system to analyze each moment precisely. This ensures accurate detection while eliminating the need for continuous real-time video processing.

- **Building a Labeled Dataset**

A dataset is developed with labeled examples of both normal activities and the specific cheating activities listed above. These labeled examples serve as the foundation for training the machine learning model.

- **Feature Extraction and Model Training**

Key features such as hand movements, interactions, and posture changes are extracted from the frames. A machine learning model is then trained to classify these activities as either normal or anomalous.

- **Detecting Anomalies**

The trained model scans video frames and flags any suspicious activities, such as a student checking answers written on their hand or passing notes.

- **Privacy-Focused and Scalable Design**

The system ensures privacy by processing each frame independently, avoiding the need for continuous live surveillance. This approach also makes the solution adaptable to various exam hall configurations. By addressing these specific activities, the system offers an effective tool to support invigilators and maintain the integrity of examinations.

1.3 Objectives of the project:

- **Analyzing Exam Videos:**

Using advanced technologies like machine learning and deep learning to review exam footage and identify any suspicious activities in the exam hall.

- **Spotting Suspicious Activities:**

Detect and classify unusual actions, such as:

Using unauthorized electronic devices.

Collaborating with others by whispering or passing items.

Odd activities like frequently leaving the seat or exchanging materials.

- **Ensuring Accurate Detection:**

Leveraging models like CNN and OpenCV for precise detection of actions and objects, aiming to reduce mistakes like missed anomalies.

- **Understanding Behavior Patterns:**

Using past data to study behavior patterns and distinguish between normal and suspicious activities, helping to make better judgments.

- **Clear and Helpful Reports:**

Providing detailed reports with a summary of detected anomalies, video clips, and key observations to help examiners take action confidently.

- **Flexible and Scalable System:**

Building a system that can handle different volumes of video data and adapt to various exam hall settings and scenarios.

- **Protecting Privacy:**

Making sure all recorded data is handled securely and ethically, with options to anonymize personal information to meet privacy standards.

1.4 Advantages

Scalability

- Can be easily scaled to monitor multiple exam halls simultaneously by adding more cameras and processing capabilities.
- Cloud-based integration allows real-time monitoring across multiple locations.

Cost-Effective

- Once implemented, reduces the need for additional human resources for invigilation.
- Prevents the cost of re-conducting exams caused by widespread cheating incidents.

Objective and Unbiased Monitoring

- Removes the potential for human bias in identifying suspicious activities.
- Ensures consistent monitoring across all students.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of Literature Survey

Survey provides an overview of the existing literature as well as an analysis of the approaches used in various studies. It covers data preprocessing techniques, neural network types, and evaluation metrics used in the field. Some of the survey papers are given below:

[1] In their paper titled “Detection of Anomalous Behavior in an Examination Hall Towards Automated Proctoring” (2017 IEEE Conference), Neha Soman, Renuka Devi M.N, and Gowri Srinivasa presented a system aimed at reducing cheating in exams by analyzing video footage to detect suspicious activities. The system relied on a K-Nearest Neighbor (KNN) classifier and Histogram of Oriented Gradients (HOG) for identifying and classifying behaviors. While it was effective at recognizing pre-defined behaviors and offered a step forward in automated proctoring, the system had its limitations. It struggled in crowded exam settings, where overlapping actions created challenges, and it was confined to detecting only known activities. These limitations highlight the need for more advanced and flexible solutions to handle real-world complexities in exam monitoring.

[2] In their 2022 publication titled “Deep Neural Network Based Anomaly Detection for Smart Surveillance System” presented at the IEEE North Karnataka Subsection Flagship International Conference (NKCon), Shubhangi D.C., Mohammed Abdul Waheed, and their team introduced a system for real-time anomaly detection using deep learning. The system leverages YOLO, CNN, and Decision Tree models, implemented with tools like Keras and OpenCV, to classify weapons and detect fire in live video feeds from security cameras. The dataset used for training included three types of weapons—handguns, knives, and large firearms—along with images of fire, supplemented by the COCO dataset for better generalization. The system demonstrates the ability to accurately identify weapons and detect unpredicted fire in real time. Despite its effectiveness, the system faces challenges in handling occlusions, low-resolution footage, and varying lighting conditions, which are common issues in real-world surveillance scenarios.

[3] In their 2023 publication titled “Video Anomaly Detection System Using Deep Convolutional and Recurrent Models” in *Results in Engineering*, Maryam Qasim and Elena Verdu proposed a system for detecting anomalies in large-scale video datasets. The system combines CNN architectures like ResNet-18, ResNet-34, and ResNet-50 with the Simple Recurrent Unit (SRU), implemented using Keras and TensorFlow, to identify normal and anomalous behaviors. Designed for real-time crime detection, it was tested on the UCF-Crime dataset, where it achieved high accuracy, precision, and AUC scores, showcasing its effectiveness for robust anomaly detection in public surveillance. However, the system has some limitations—it relies heavily on the contextual definition of anomalies, making it less adaptable to unfamiliar environments. Additionally, it struggles to detect subtle anomalies and faces high computational costs when processing large video sequences, which can be a challenge for scalability in real-world applications.

[4] In their 2023 publication titled “Real-time Video Anomaly Detection for Smart Surveillance” in *IET Image Processing*, Manal Mostafa Ali introduced a real-time anomaly detection system designed for smart surveillance applications. The system integrates background subtraction using a Mixture of Gaussians, convolutional autoencoders, and YOLOv5 for object detection. By isolating and analyzing regions of interest, it effectively identifies anomalies in video streams, achieving an impressive average AUC of 94.94% on benchmark datasets such as UCF Crime, UCSD Ped2, UCSD Ped1, CUHK Avenue, and StreetScene. Despite its effectiveness, the system has some challenges—it is context-dependent, with its performance affected by lighting conditions, noise, and occlusions. Additionally, the high computational costs for frame reconstruction and a tendency to produce false positives due to variations in normal patterns pose limitations for real-world deployment. It also struggles with detecting subtle anomalies, highlighting areas for further improvement.

[5] In their 2021 paper “A Comprehensive Review on Deep Learning-Based Methods for Video Anomaly Detection” published in *Image and Vision Computing*, Rashmiranjan Nayak, Umesh Chandra Pati, and Santos Kumar Das explored various techniques for identifying anomalous activities in video streams. The review categorizes video anomaly detection methods into four main types: trajectory-based, global pattern-based, grid pattern-based, and hybrid approaches.

It highlights the use of different training frameworks—supervised, unsupervised, and semi-supervised—and discusses the prominent use of models such as autoencoders, convolutional neural networks (CNNs), and spatiotemporal networks. The paper also reviews benchmark datasets like UCSD Ped1 and Ped2, Subway, U-turn, UMN, Avenue, UCFcrime, and CUHK Avenue. Despite the promise of these methods, the review identifies challenges, including the dependency on high-quality datasets, the lack of standardization in defining anomalies, and difficulties with computational efficiency, particularly in real-time applications. These factors make it challenging to deploy anomaly detection systems effectively in diverse, dynamic environments.

[6] In their 2023 paper “Deep Learning-Based Anomaly Detection in Video Surveillance” published in *Sensors*, Huu-Thanh Duong, Viet-Tuan Le, and Vinh-Truong Hoang reviewed various deep learning methods for anomaly detection in surveillance videos. The paper categorizes these approaches into four main types: reconstruction-based, multiclass classification, future frame prediction, and scoring. The review also discusses the use of generative adversarial networks (GANs), CNNs, Histogram of Gradients (HOG), and the Speed-up Robust Feature (SURF) algorithm in detecting anomalies. Tested on several benchmark datasets like the CASIA Action Database, Subway Database, UMN Crowd Abnormality Database, and Avenue Database, the methods show promise in enhancing video surveillance systems. However, the paper highlights several challenges, including high computational costs, scene dependency, noise, occlusion, low illumination, and dataset imbalances. Additionally, the complexity of implementing these techniques in real-time applications remains a significant hurdle. These issues underline the need for further improvements to make anomaly detection systems more effective and efficient in real-world scenarios.

[7] In their 2024 paper “Deep Learning-Based Anomaly Detection in Real-Time Video” published in *Multimedia Tools and Applications*, Ahmed Elmetwally, Reem Eldeeb, and Samir Elmougy proposed an automated system for detecting anomalies in real-time video feeds. The system utilizes the I3D-ResNet50 model for feature extraction and a weakly supervised scoring mechanism through Multiple Instance Learning (MIL). Tested on datasets such as UCF-Crime, UCFCrime2Local, UCSD-Peds, and XD-Violence, the system demonstrates its capability in

detecting unusual events in video streams. However, it faces challenges in handling low-resolution anomalies and the variability in characteristics that can occur in different environments or video quality. Despite these hurdles, the approach presents a promising solution for real-time anomaly detection in surveillance systems.

[8] In their 2019 IEEE Conference publication “Deep Learning-Based Anomaly Detection on Surveillance Videos: Recent Advances”, Akmalazakia Fatan Derana Marsiano, Indah Soesanti, and Igi Ardiyanto discuss the growing need for computer vision systems to replace human monitoring in surveillance, with the goal of detecting suspicious activities in real-time. The paper reviews various deep learning techniques, including variants of CNNs like ConvNet 3D and Temporal 3D ConvNets (T3D), as well as models like the Gaussian Mixture Model (GMM) and Graph Convolutional Label Noise Cleaner (GCN). These methods are designed to capture both motion and spatial data effectively. The authors tested their approaches on benchmark datasets such as UCF-Crime, HMDB-51, and UCF-101. However, they point out several challenges, including the need for computing efficiency, the requirement for large amounts of labeled data, and difficulties in processing untrimmed video data with long-range temporal dependencies. Despite these challenges, the paper highlights the potential of these deep learning methods to significantly improve anomaly detection in surveillance systems.

[9] In their 2017 paper “Abnormal Behavior Recognition for Intelligent Video Surveillance Systems” published in Expert Systems with Applications, Amira Ben Mabrouk and Ezzeddine Zagrouba provide a comprehensive review of methods used to detect abnormal behaviors in video surveillance. The paper emphasizes two main stages: behavior representation and behavior modeling. In the first stage, behavior representation involves feature extraction to identify objects of interest in a scene. In the second stage, behavior modeling uses classification techniques to determine whether a behavior is abnormal. The authors discuss various techniques such as the geodesic graph, SVM classifiers, Markov Random Field (MRF) models, Gaussian Mixture Models (GMM), and Hidden Markov Models (HMM) for both stages. The review is based on datasets like UCSD, Violent Flows, Action Movies, Hockey, and Web datasets. While the methods discussed show promise, the paper also highlights several challenges, including the difficulty in selecting robust features, handling scene changes and occlusions, and the need for large datasets to train deep learning models effectively.

2.2 Literature Survey Summary

This survey provides valuable insights that can guide the development of our anomaly detection system. By analyzing proven techniques, we can adopt effective feature extraction methods like CNN architectures or HOG and utilize hybrid models such as CNN with RNN or YOLO for robust anomaly detection. Addressing common challenges identified in the literature, such as occlusions and crowded settings, we can incorporate models that combine spatial and temporal analysis to enhance performance in complex scenarios. The survey also emphasizes the importance of diverse and well-labeled datasets, which aligns with our focus on preprocessing (e.g., resizing, grayscale conversion) to ensure compatibility with our system. To achieve real-time efficiency, we can leverage lightweight models like YOLOv5 while balancing accuracy and computational cost. Additionally, adopting standardized metrics like accuracy, precision, and AUC allows for rigorous evaluation and benchmarking against dataset. By addressing unresolved challenges, such as subtle anomaly detection and adaptability to dynamic environments, we can develop an innovative and effective solution tailored to our project's needs. This approach ensures our system is both practical and contributes meaningfully to advancements in anomaly detection technology.

CHAPTER 3

REQUIREMENT SPECIFICATIONS

3.1 Non-Functional Requirements

Functional requirements define the core operations and tasks the system must perform to achieve its objectives. These include tasks related to data handling, model training, prediction, and video annotation, ensuring the system delivers its intended functionality effectively.

3.1.1 Performance

Definition: Performance refers to the speed and efficiency with which the system performs its tasks, including processing and response times.

Implementation:

- The system achieves efficient video frame processing, maintaining a rate of 15-30 frames per second during inference on a CPU. Optimized preprocessing and model architecture enable this performance.
- The training process is structured to complete within a reasonable timeframe by employing techniques such as batch processing and early stopping, tailored to the dataset size and hardware configuration.

3.1.2 Scalability

Definition: Scalability is the system's ability to handle increasing workloads, such as larger datasets or higher-resolution inputs, without significant reconfiguration or degradation in performance.

Implementation:

- The system can adapt to varying numbers of classes and datasets with minimal adjustments.
- High-resolution videos are supported by dynamically resizing frames during preprocessing to ensure compatibility while preventing crashes.

3.1.3 Usability

Definition: Usability ensures that the system is intuitive, user-friendly, and accessible, enabling users to easily interact with its features.

Implementation:

- The interface, developed using Streamlit, provides a smooth user experience with clear options for uploading videos, viewing results.
- Navigation tabs, such as Home, Detection, and About, guide users through the system's functionalities seamlessly.

3.1.4 Reliability

Definition: Reliability is the consistency and accuracy of the system in performing its intended tasks without errors or interruptions.

Implementation:

- Consistent predictions are ensured by fixing random seeds during model training and testing, enhancing reproducibility.
- Annotated output videos are generated without frame loss or corruption, ensuring accurate representation of detected anomalies.

3.1.5 Portability

Definition: Portability refers to the system's ability to operate across different platforms and environments without requiring significant changes.

Implementation:

- The system is platform-independent, running on any machine with Python and the required libraries installed.
- It supports both GPU-enabled and CPU-only environments, detecting and utilizing available resources automatically.

3.1.6 Security

Definition: Security ensures that the system safeguards sensitive data and maintains confidentiality during operations.

Implementation:

- All video processing is performed locally, ensuring that sensitive data, such as student identities and exam hall footage, remains private.

- No external data sharing or cloud-based processing is involved, upholding data security standards.

3.1.7 Resource Utilization

Definition: Resource utilization refers to the efficient use of system resources, such as memory, processing power, and hardware capabilities.

Implementation:

- Memory and processing power are efficiently utilized by optimizing batch sizes and data loading during training and inference.
- Available GPUs are utilized for computation-intensive tasks, accelerating training and inference without overloading the CPU.

3.1.8 Accuracy

Definition: Accuracy measures the system's ability to produce correct and reliable outputs, reflecting the quality of its predictions.

Implementation:

- The CNN model is fine-tuned to achieve a baseline accuracy of over 92% on training and validation datasets, with techniques like data augmentation and hyperparameter optimization improving generalization.
- Confidence scores are provided alongside predictions, offering meaningful insights into the reliability of the results.

3.2 Functional Requirements

Non-functional requirements outline the system's performance, reliability, and usability standards. They ensure that the system not only works as intended but also meets quality benchmarks such as efficiency, scalability, and privacy.

3.2.1 Dataset Management:

- The system must be capable of loading and organizing images from a structured dataset, where each class (e.g., "copying" or "normal") is stored in a dedicated subdirectory.
- It must preprocess the images by resizing, normalizing, and applying data augmentation techniques such as flipping, rotation, and zooming to enhance the training process.

3.2.2 Model Training:

- The system must train a Convolutional Neural Network (CNN) using the prepared dataset and automatically save the trained model in a reusable format (e.g., .h5).

3.2.3 Frame Classification:

- The system must process video frames individually, classify each frame into one of the predefined categories (e.g., “using mobile,” “peeking,” “normal”), and determine the confidence score of each prediction.

3.2.4 Video Annotation:

- The system must annotate video frames with the predicted class label and the corresponding confidence score.
- It must save the annotated video in a specified output format (e.g. .mp4v or .avi).

3.2.5 Progress Reporting:

- The system must provide feedback during operations, such as indicating training progress, the current status of video processing, and when the output file is saved.

3.2.6 Error Handling:

- The system must handle exceptions gracefully, such as missing dataset folders, invalid video files, or unsupported codecs, and provide meaningful error messages to guide the user.

3.2.7 Output Generation:

- The system must produce and save an annotated video as the final output, along with detailed logs or messages summarizing the operation.

3.3 Software Requirements

Software requirements outline the key attributes and expectations of the system, including its performance, usability, scalability, and maintainability. These ensure the system operates effectively, meets user needs and remains adaptable.

3.3.1 Operating System: Windows 10 or 11

3.3.2 Python Environment:

- Python 3.8+.
- Virtual environment for dependency management (e.g., venv or conda).

3.3.3 Libraries/Packages:

- Tensorflow (2.x or above).
- OpenCV-python.
- NumPy.
- Keras
- Scikit-learn.
- Streamlit

3.3.4 Integrated Development Environment (IDE):

- Visual Studio Code, PyCharm, or Jupyter Notebook.

3.3.5 Video File Format:

- Input: .mp4, .avi (or similar supported format).
- Output: .mp4 or another suitable format with specified codecs.

3.4 Hardware Requirements

3.4.1 GPU: A GPU is highly recommended for training deep learning models, especially for tasks involving large datasets like diabetic retinopathy detection.

- NVIDIA GPUs (such as RTX 3060, 3080, or higher) with CUDA support are preferable.
- At least 8 GB of VRAM, but more is better for larger models and datasets.

3.4.2 CPU: A multi-core processor is recommended for data preprocessing and handling other computational tasks.

- Intel i7 or higher.

3.4.3 RAM: Adequate RAM is needed to handle large datasets and multiple processes.

- Minimum 16 GB, but 32 GB or more is preferable for better performance.

3.4.4 Storage: Sufficient storage for datasets, models, and other files.

- SSD (Solid State Drive) with at least 512 GB. More storage may be needed depending on the size of the dataset.

3.5 Dataset Requirements

3.5.1 Structure:

- Organized into subdirectories representing each class (e.g. Normal, using mobile, etc.).

3.5.2 Data Distribution:

- Ensure balanced data across all classes to prevent model bias.
- Include a validation split (e.g., 20%) for proper model evaluation.

3.5.3 File Format:

- Images should be in a readable format like .jpg or .png.
- Video files for inference should follow a consistent resolution and frame rate.

CHAPTER 4

PROPOSED MODEL

4.1 Problem Statement

Maintaining fairness during exams can be challenging with traditional monitoring methods like invigilators or CCTV, which are often error-prone and invasive. This project aims to develop an anomaly detection system for offline exam monitoring to identify suspicious activities, like cheating or unusual activities, by analyzing video footage using deep learning techniques.

4.2 Proposed solution

The proposed solution aims to leverage Convolutional Neural Networks (CNNs) to detect anomalous activity in an exam hall setting by analysing visual data. The goal is to develop a model capable of classifying images into various categories, including identifying potential cheating or other irregularities, using images captured during the exam.

4.2.1. Dataset Preparation and Preprocessing:

- i. **Data Collection:** The system requires a dataset of images (or frames extracted from videos) captured in the exam hall. These images are labeled according to various scenarios, such as normal, suspicious activity, or cheating. These images are then organized into respective directories based on these labels.
- ii. **Frame Extraction Using OpenCV:** Video footage from the exam hall is processed using the OpenCV library to extract individual frames at regular intervals. These frames form the primary dataset for training the model. Extracted frames are saved in directories labeled according to specific activities(e.g., “normal,” or “cheating”).
- iii. **Preprocessing:** The images are preprocessed by resizing them to a standard resolution of 128x128 pixels, normalizing the pixel values to the range [0, 1], and applying data augmentation techniques (such as rotation, zoom, and horizontal flipping) to improve the generalization of the model.

4.2.2. Model Architecture:

- i. **CNN Structure:** The model is constructed using a series of convolutional layers designed to learn key features from the images. These layers are followed by pooling layers that reduce the dimensionality of the feature maps, retaining the most important

information. The final layers are fully connected (Dense) layers that classify the images into one of the predefined classes.

- ii. **Dropout Regularization:** To mitigate the risk of overfitting, a dropout technique is employed in the dense layer, where a portion of the neurons is randomly ignored during training. This ensures that the model generalizes well to unseen data.

4.2.3. Model Training:

- i. **Training-Validation Split:** The dataset is split into training and validation sets, allowing the model to learn from the training data while being evaluated on the validation set to monitor for overfitting.
- ii. **Optimization:** The model is compiled using the Adam optimizer for efficient learning, with categorical cross-entropy as the loss function to handle multi-class classification. The model is trained for 30 epochs to ensure that it converges to an optimal solution.

4.2.4. Anomaly Detection:

- i. **Prediction:** After training, the model is deployed to classify new images from exam hall video feeds. The model assigns each image to one of the predefined categories. Anomalous activities, such as cheating or unauthorized activities, can be detected by classifying the image under a specific anomaly category (e.g., copying, discussing).
- ii. **Confidence Scoring:** The model generates a confidence score for each prediction, indicating the certainty of the classification. If the score for an anomalous activity exceeds a predefined threshold, the system can trigger an alert for further review.

4.2.5. Reporting and Monitoring:

Logging: Each time an anomaly is detected, the system logs the event, including type of anomaly, and confidence level of the model's prediction. These logs provide valuable insights for exam hall supervisors to review suspicious activities.

4.2.6. Deployment and Continuous Improvement:

- i. **Model Deployment:** Upon successful training, the model is saved as a .h5 file . The model continuously classifies the frames and flags any suspicious activity as anomalies.
- ii. **Retraining:** As new data becomes available, the system can be periodically retrained with updated images to adapt to evolving suspicious activities in the exam hall, ensuring continuous improvement of the detection capabilities.

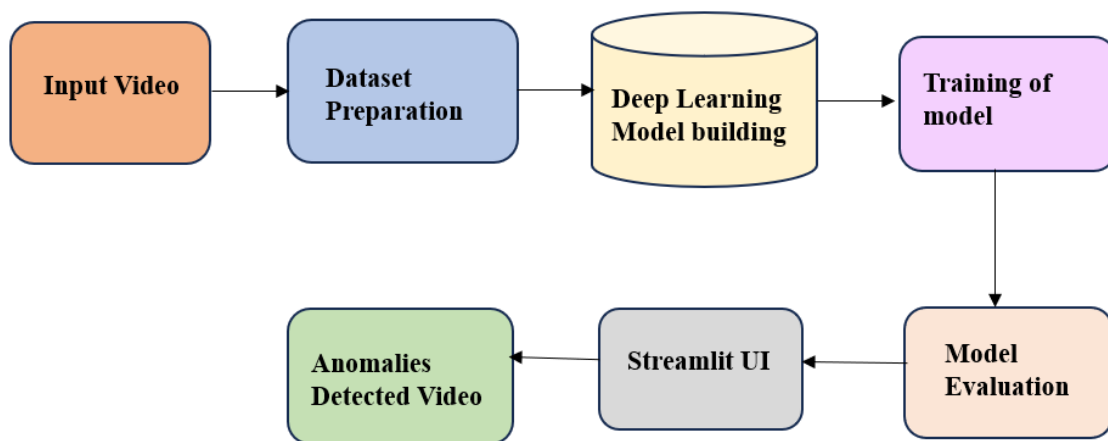


Fig 4.1 Working process of proposed methodology

This figure represents a pipeline for detecting anomalies in videos using a deep learning-based framework. a step-by-step explanation of the flow:

- i. **Input Video:**
 - The process starts with the input video, which serves as the raw data for analysis.
- ii. **Dataset Preparation:**
 - The input video undergoes preprocessing, which may include tasks like frame extraction, resizing, normalization, augmentation or noise reduction to prepare the data for further analysis.
- iii. **Deep Learning Model Building:**
 - The pre-processed data is fed into a module where a deep learning model is built. This step typically involves selecting and designing a neural network architecture suitable for anomaly detection.
- iv. **Training of Model:**
 - The augmented data is used to train the deep learning model. During training, the model learns patterns and features to distinguish normal from anomalies.

v. Model Evaluation:

- After training, the model's performance is evaluated using test data. Metrics like accuracy, precision, recall, and F1-score may be used to assess the model's effectiveness.

vi. Streamlit UI:

- A user interface built with Streamlit is implemented for interaction. It allows users to upload videos, view results, and analyse anomalies.

vii. Anomalies Detected Video:

- The Streamlit UI outputs the video with anomalies marked or highlighted for easy identification and review.

This pipeline provides an end-to-end solution for video anomaly detection, combining deep learning with a user-friendly interface for practical application.

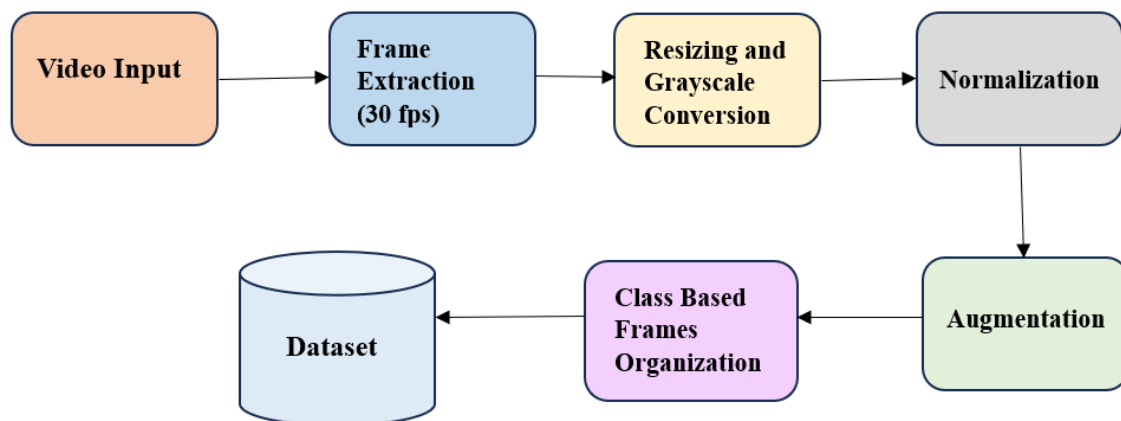


Fig 4.2 Working process of dataset preparation

This figure outlines the preprocessing steps for a video dataset. Frames are extracted from the video, resized, converted to grayscale, normalized, and augmented to enhance the dataset. Finally, frames are organized into classes, creating a structured dataset for training.

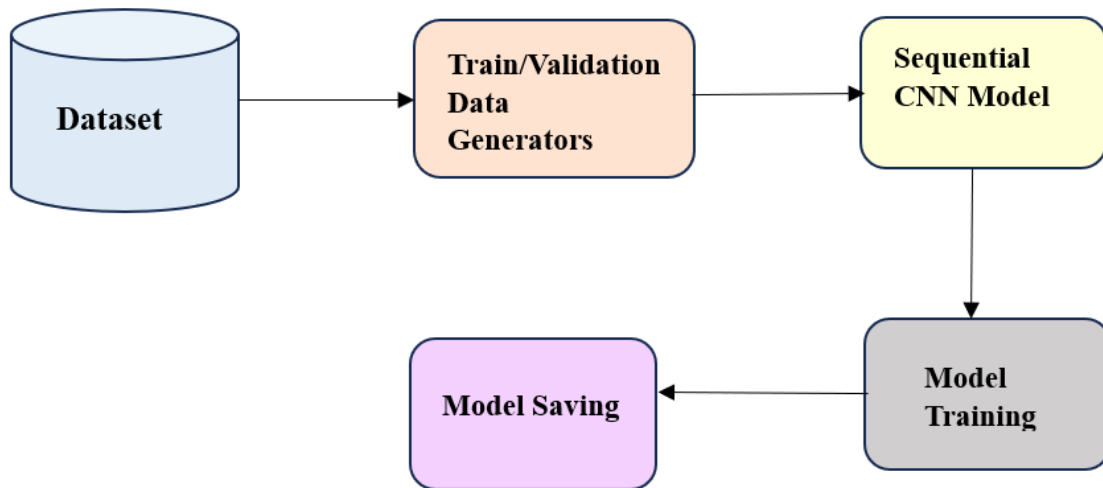


Fig 4.3 Working process of model training

This figure shows the model training process. The dataset is passed through data generators for training and validation, then fed into a Convolutional Neural Network (CNN) model. The trained model is saved after successful training for future use.

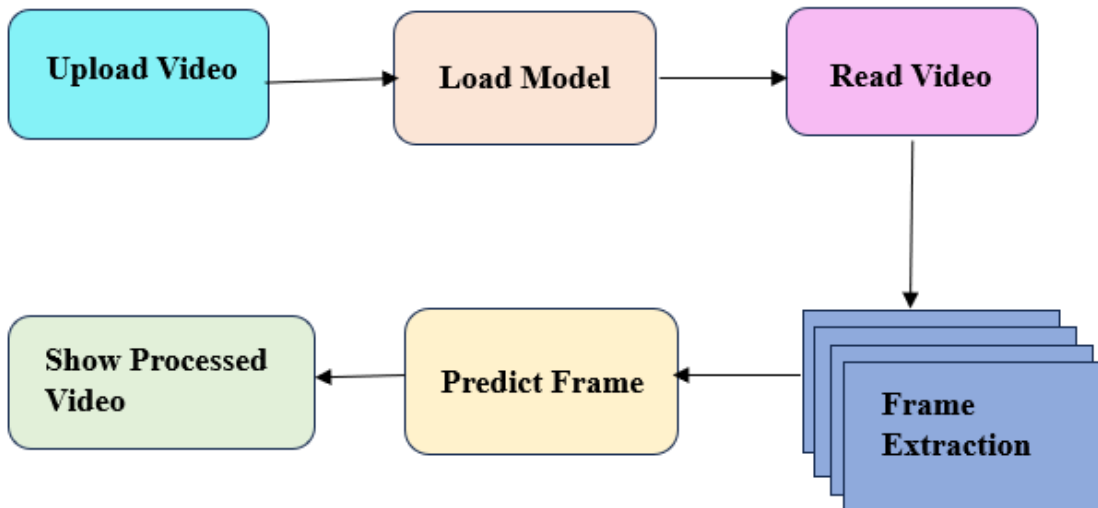


Fig 4.4 Working process of streamlit UI

This figure depicts the workflow for video anomaly detection. A video is uploaded, and a pre-trained model is loaded to analyse it. The video frames are processed and predicted for anomalies, and the results are displayed as a processed video.

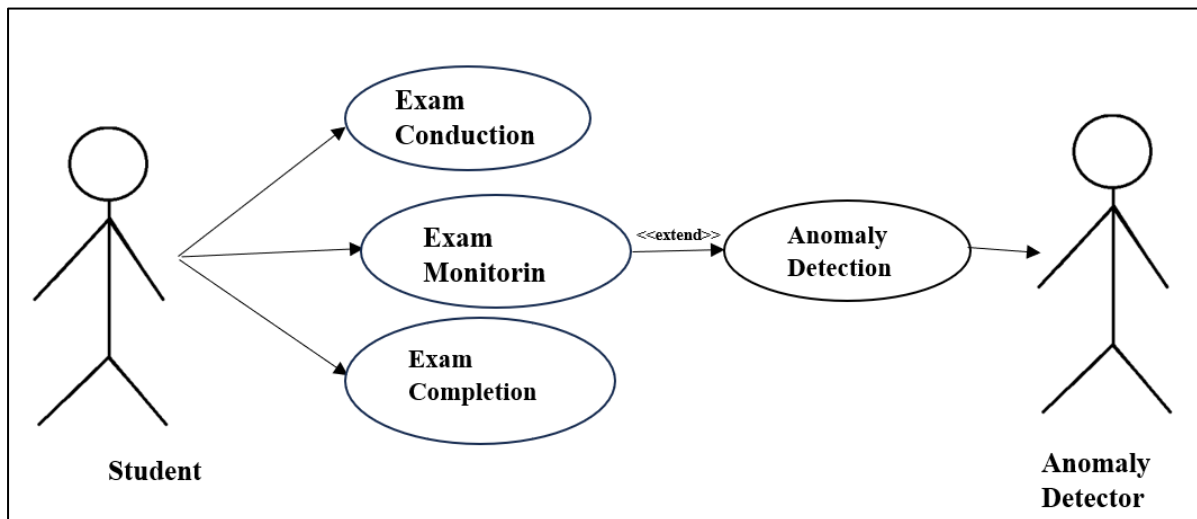


Fig 4.5 Use case diagram for anomaly detection

Use case diagram is nothing but how the user interacts with the system in this particular use case diagram video is given as input and the fig 4.5 is represents the interaction between a user and the video anomaly detection system. The user uploads a video, which is then processed to predict anomalies. Finally, the system provides the user with a processed video highlighting the detected anomalies.

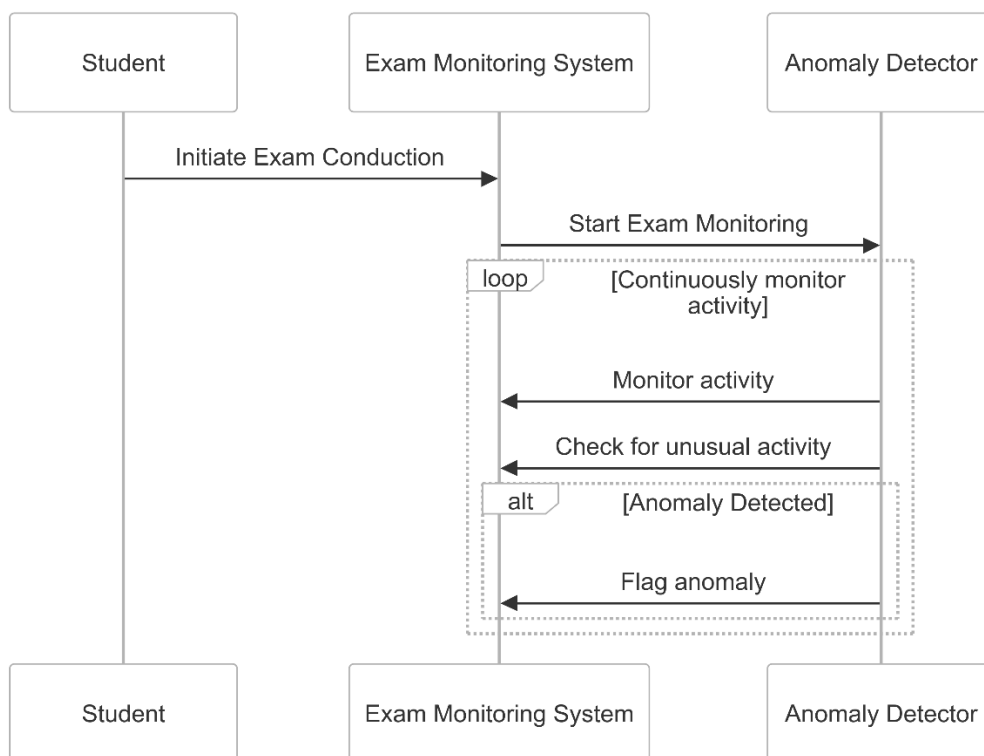


Fig 4.6 Sequence diagram related to use case

This sequence diagram represents the interaction between a Student, the System, and an Anomaly Detector during an online exam. The process begins with start of exam after which the system starts monitoring their activities. The system continuously observes and checks for any unusual activities by communicating with the anomaly detector. If an anomaly is detected, the system highlights it for further review.

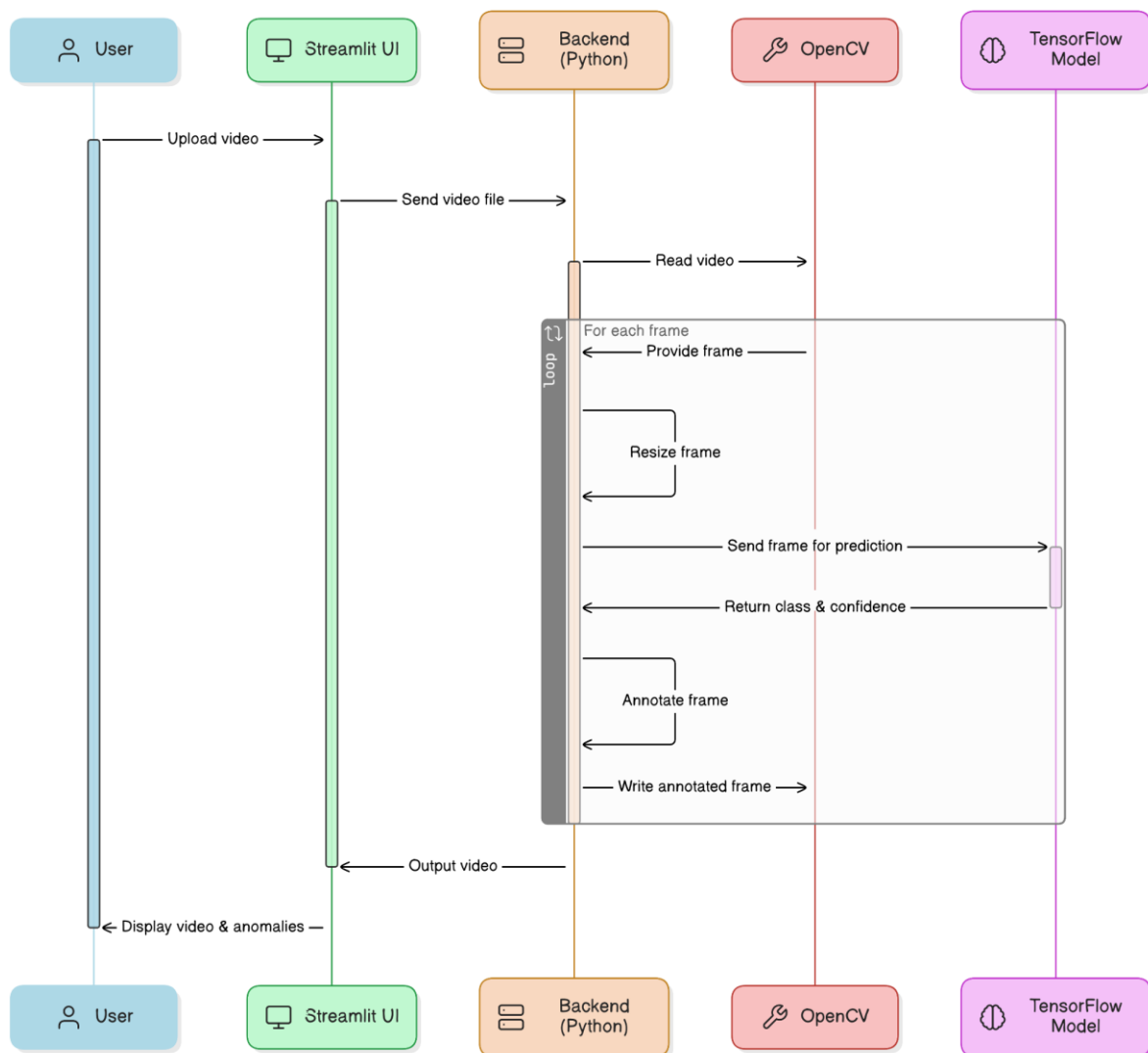


Fig 4.7 Sequence diagram for user interface

This diagram represents the workflow of a video anomaly detection system. It shows the interaction between different components, including the user, a Streamlit-based UI, a Python backend, OpenCV, and a TensorFlow model. Here's the detailed explanation:

Components

- **User:**
The end-user who uploads the video file for anomaly detection.
- **Streamlit UI:**
A user interface built using Streamlit where users can upload video files and view the processed results.
- **Backend (Python):**
The processing engine written in Python, video analysis and anomaly detection tasks.
- **OpenCV:**
A library used for video processing, including frame extraction and annotation.
- **TensorFlow Model:**
A pre-trained machine learning model that performs anomaly detection by predicting classes and confidence for each frame.

Process Workflow

- **Video Upload:** The user uploads a video through the Streamlit UI.
- **Send Video File:** The Streamlit UI forwards the uploaded video file to the backend for processing.
- **Read Video:** The backend uses OpenCV to read the video file and extract individual frames for analysis.
- **Frame-by-Frame Processing (Loop):**
 - **Provide Frame:** Each frame from the video is provided for further processing.
 - **Resize Frame:** Frames are resized to meet the input requirements of the TensorFlow model.
 - **Prediction:**
 - The resized frame is sent to the TensorFlow and CNN model.
 - The model predicts the class (e.g., normal or anomaly like copying , chit passing , discussing etc.) and the confidence score.

- **Annotate Frame:**

The backend annotates the frame with the predicted class and confidence score using OpenCV.

- **Write Annotated Frame:**

Annotated frames are written to a new output video file.

- **Output Video:**

The processed video, including annotations of anomalies, is sent back to the Streamlit UI.

- **Display Results:**

The Streamlit UI displays the output video along with highlighted anomalies for the user to review.

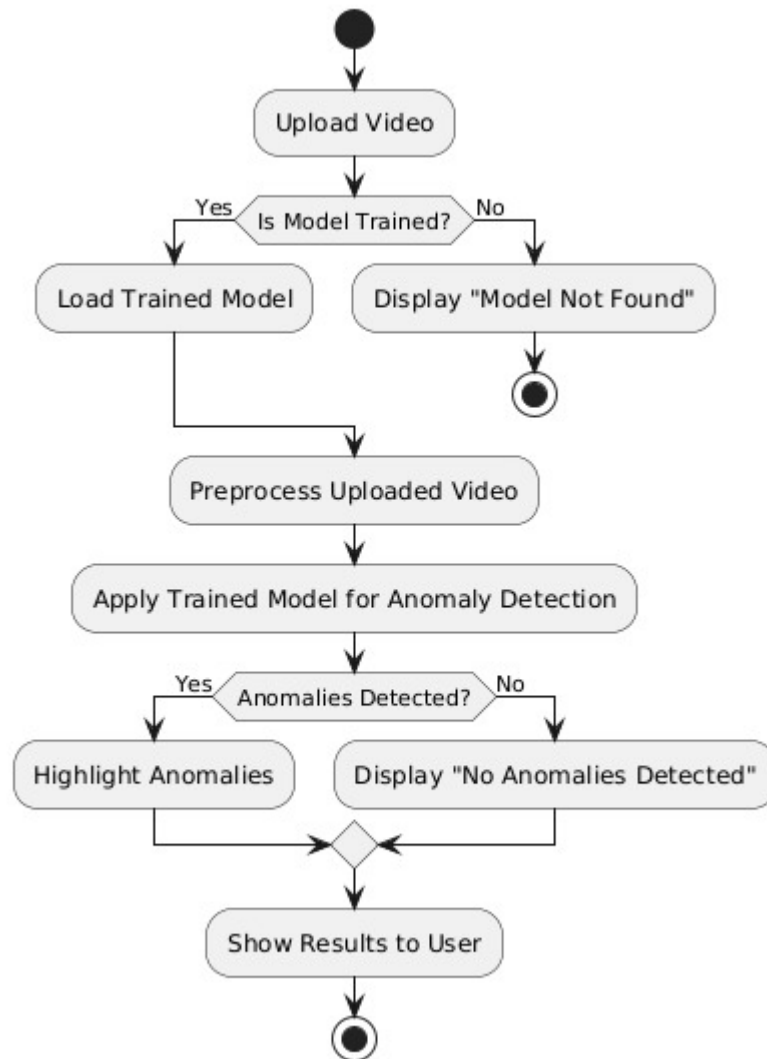


Fig 4.7 Activity diagram for user interface

The diagram represents a process for anomaly detection in uploaded videos. It first checks if a trained model exists; if not, it displays an error. If a model is found, the video is preprocessed, analyzed for anomalies, and the results (anomalies highlighted or "no anomalies detected") are shown to the user.

CHAPTER 5

DATASET PREPARATION

Purpose: The dataset is specifically created for focusing on the detection of anomalies in offline exam monitoring. It aims to support the training and evaluation of machine learning models for accurate anomaly detection in video-based data.

5.1 Data Preparation and Preprocessing:

Preprocessing is a crucial step in preparing video data for machine learning models. It ensures consistency in the data format, reduces noise, and enhances model performance. This section details the preprocessing pipeline applied to the input video data.

Steps in Preprocessing

5.1.1 Frame Extraction

Objective: Convert the video into individual frames for further processing.

Process:

- Use OpenCV to read the video.
- Extract frames at a fixed interval (e.g., 30 frames per second).

Visualization:

Video Input



Some Extracted Frames



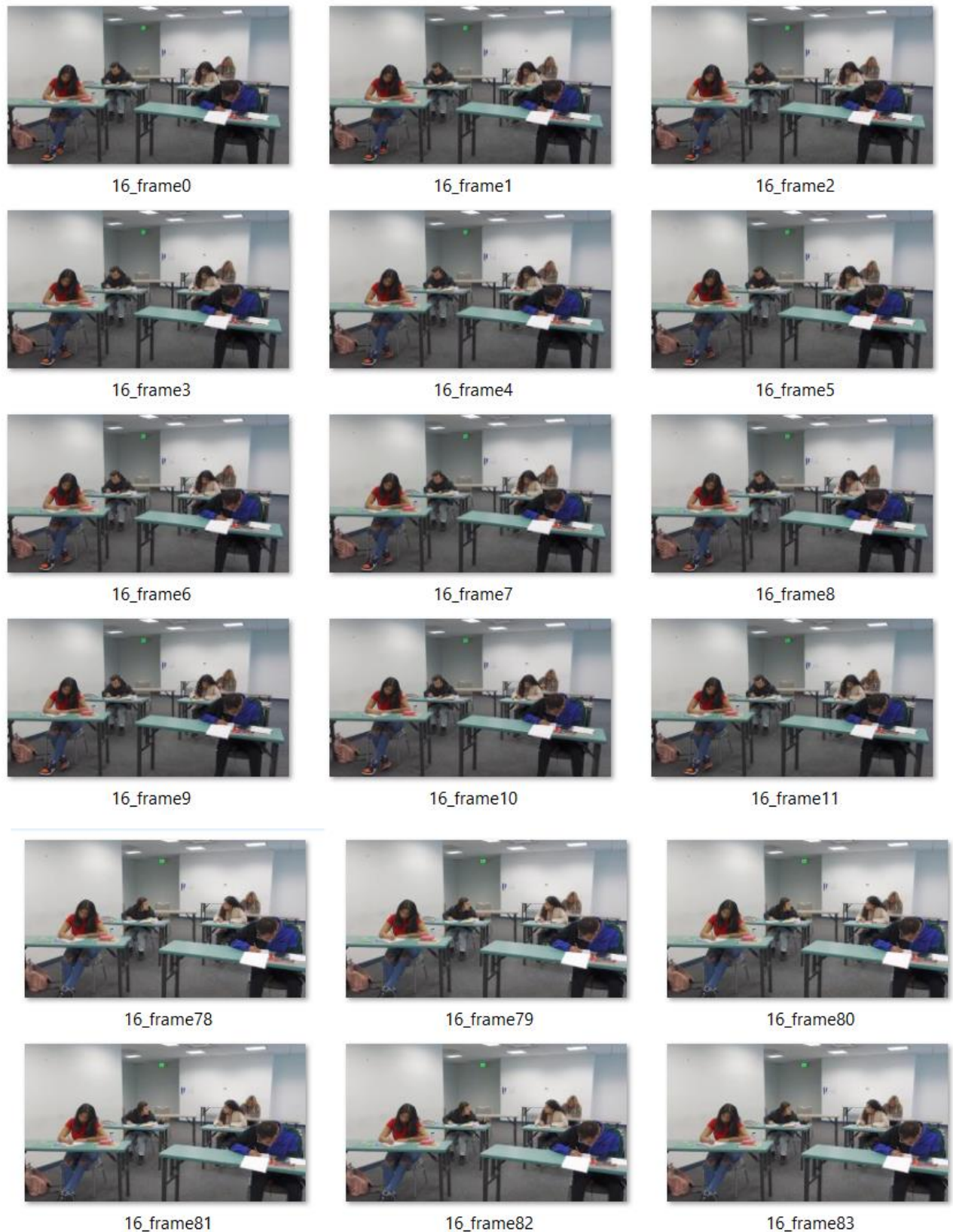


Fig 5.1 Frame extraction for exam video

5.1.2 Resizing and Grayscale Conversion

Objective:

Ensure uniform frame dimensions (e.g., 128x128 pixels) and convert the frames to grayscale for compatibility with the model.

Process:

- Resize each frame to a standard dimension of 128x128 pixels using the PIL (Python Imaging Library).
- Convert the resized frames to grayscale to reduce the computational complexity and improve model performance.

Visualization:

Original Frames

Original Frame 0



Original Frame 1



Original Frame 2



Resized and Gray scaled

Processed Frame 0



Processed Frame 1



Processed Frame 2



Original Frame 3



Original Frame 4



Original Frame 5



Original Frame 6



Original Frame 7



Processed Frame 3



Processed Frame 4



Processed Frame 5



Processed Frame 6



Processed Frame 7



Fig 5.2 Resizing and gray scaling of exam video frames

5.1.3 Normalization

Objective: Scale pixel values to the range $[0, 1]$ to improve model convergence.

Process:

Convert pixel intensity values from 0-255 to floating-point values in the range $[0, 1]$.

Visualization:

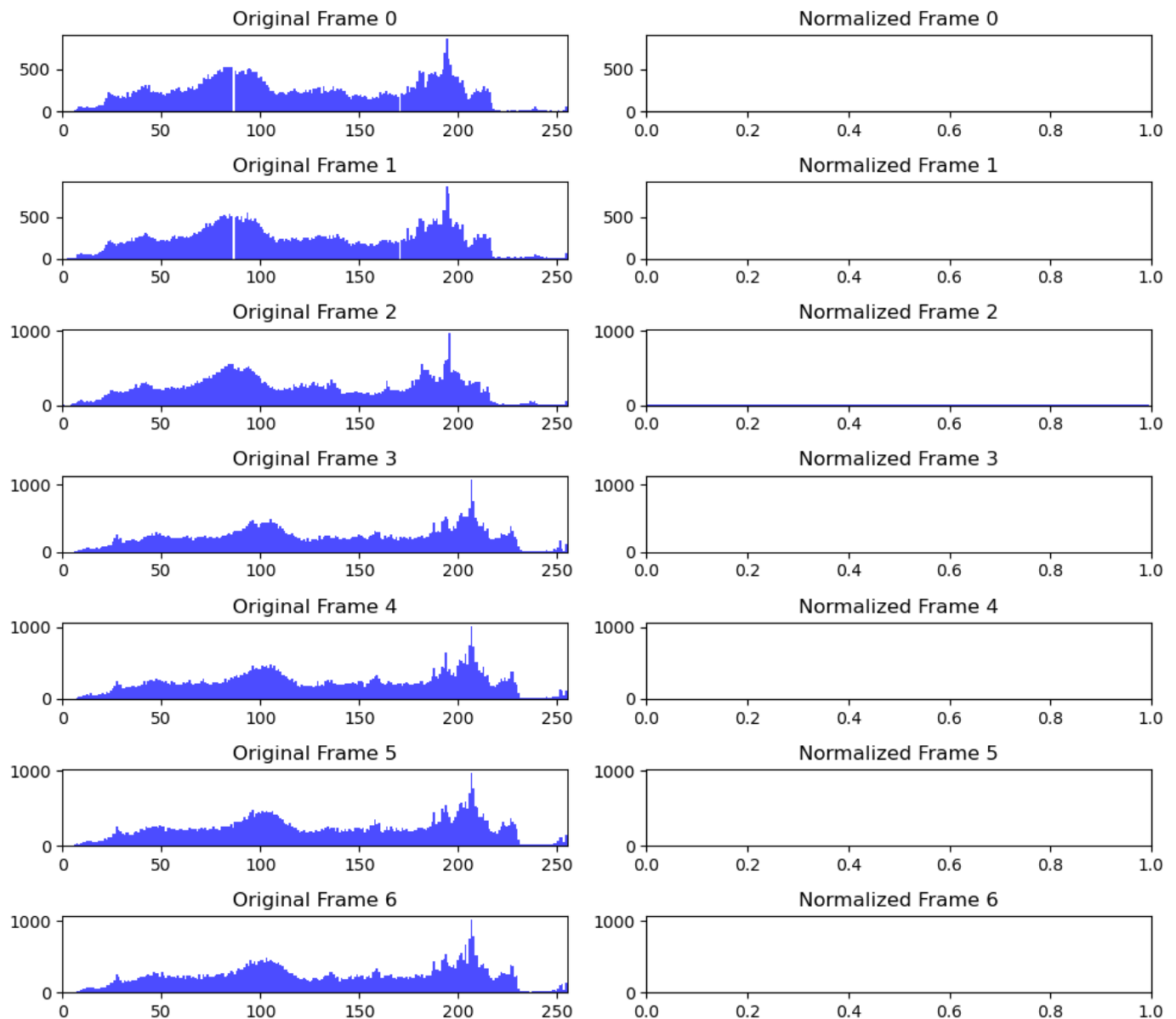


Fig 5.3 Normalization of exam video frames

5.1.4 Data Augmentation

Objective: Increase dataset diversity to improve model robustness.

Process: Augmentations applied: Flipping, rotation, and brightness adjustments.



Fig 5.4 Augmented frames of exam video

5.2 Data Organization

5.2.1 Class Selection and Folder Creation:

- i. **Class Selection:** Identify the 9 distinct activities from the video data that need to be categorized: normal, peeking, discussing, cheat_passing, using_copy_cheat, using_mobile, suspicious, ,copying showing_answer.
- ii. **Folder Creation:** Create separate folders for each of the identified classes to store frames corresponding to each activity.

5.2.2 Frame Categorization and Naming:

- i. **Manual Frame Categorization:**
 - Inspect each frame extracted from the video and manually assign it to the appropriate class based on observed activity.
 - For example, a frame showing suspicious activity is placed in the suspicious folder.
- ii. **Frame Naming:**
 - Each frame is renamed using the format {class_name}_{frame_number}.jpg, ensuring clear identification and association with the specific activity.
 - Example: suspicious_1.jpg.
- iii. **Purpose:**
 - This organization ensures that the dataset is labelled and structured in a way that is coherent and suitable for training a CNN model.
 - It enhances the model's accuracy by providing well-labelled and categorized data, reducing ambiguity and improving generalization.

5.2.3 Dataset Structure :

/Dataset

```

├── normal
│   ├── normal_0.jpg
│   ├── normal_1.jpg
│   └── ...
├── peeking
│   ├── peeking_0.jpg
│   ├── peeking_1.jpg
│   └── ...
├── discussing
│   ├── discussing_0.jpg
│   ├── discussing_1.jpg
│   └── ... et al.

```





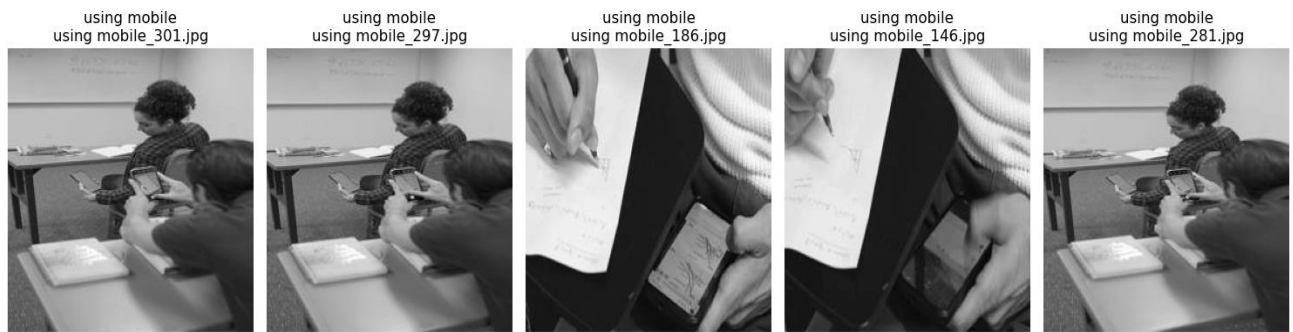


Fig 5.5 Visualizing sample images of exam videos

5.3 Outputs

5.3.1 Content: The dataset contains pre-processed frames extracted from exam video recordings, representing nine distinct activities categories:

1. Chit passing
2. Copying
3. Discussing
4. Normal
5. Peeking
6. Showing answer
7. Suspicious
8. Using copy chit
9. Using mobile

Each frame is labelled and manually curated to ensure high-quality data. Frames are scaled, resized, converted to grayscale, and duplicates are eliminated to maintain uniqueness and compatibility with CNN-based models.

5.3.2 Size:

Total Images: 3689

Class Labels	No. of Images
Chit passing	411
Copying	410
Discussing	411
Normal	411

Peeking	411
Showing answer	411
Suspicious	411
Using copy chit	411
Using mobile	411

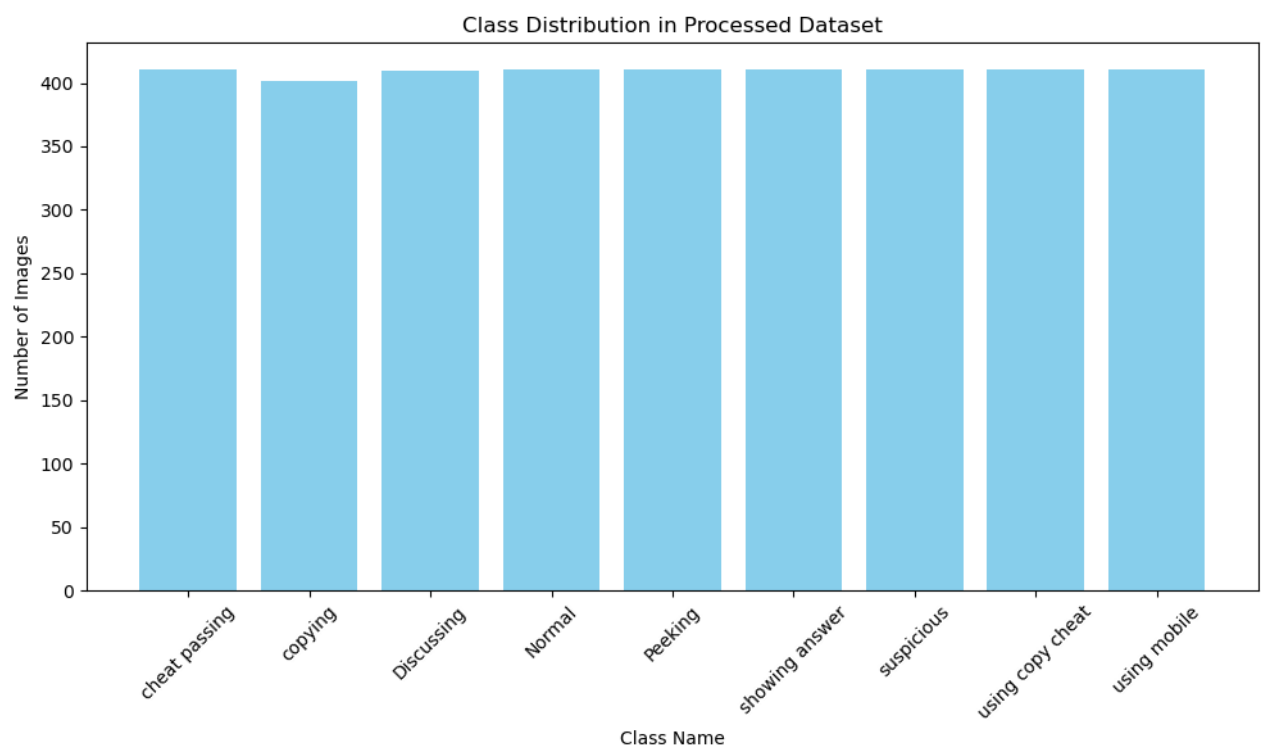


Fig 5.6 Visualizing class distribution

The bar graph illustrates the balanced distribution of image counts across nine classes in the anomaly detection dataset.

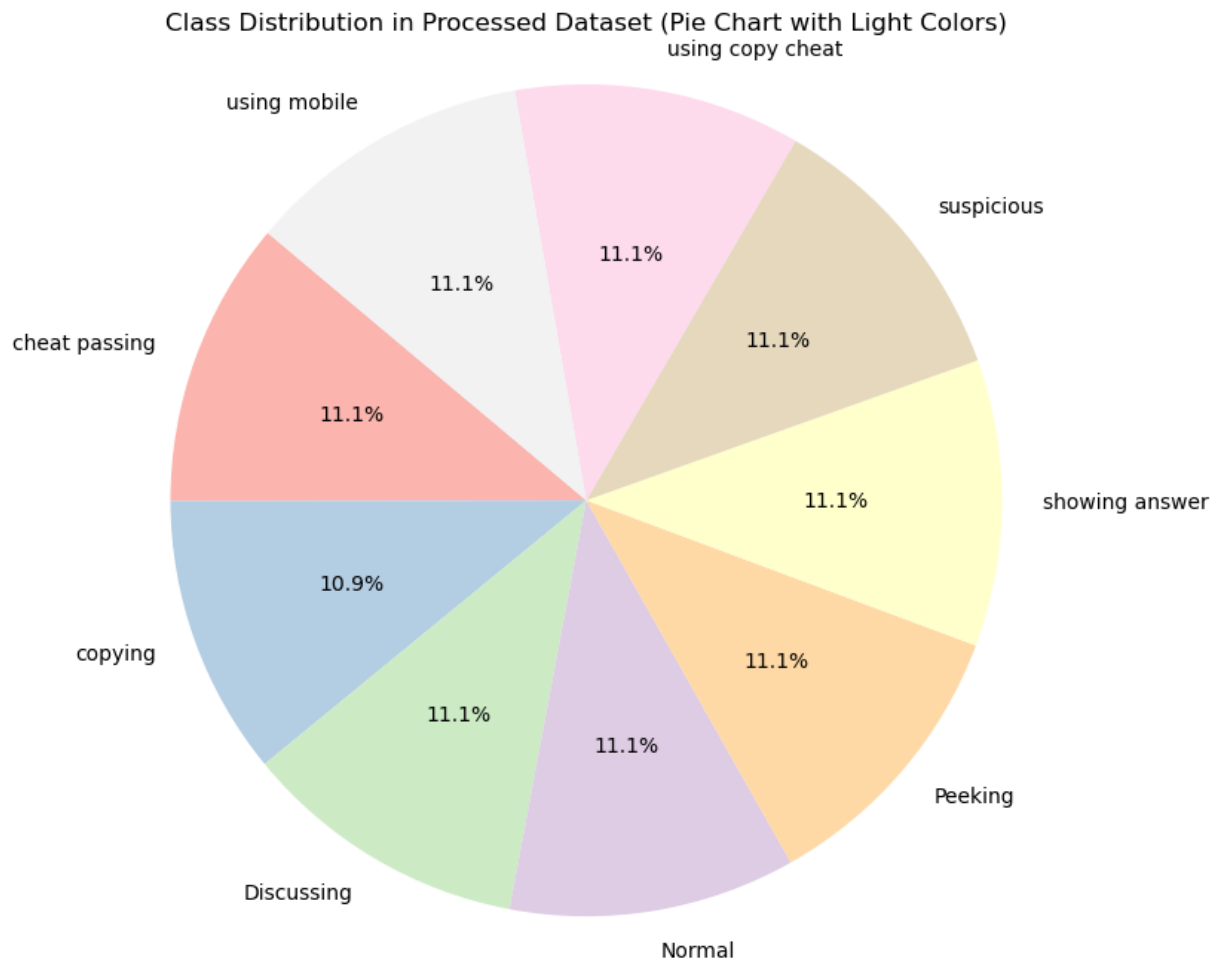


Fig 5.7 Pie chart of dataset distribution

The pie chart display the percentage distribution of images across nine classes in the dataset.

CHAPTER 6

IMPLEMENTATION

In this project, various algorithms and tools were employed to build an anomaly detection system for offline exam monitoring. The system analyzes video footage to identify activities such as "Using Mobile," "Copying," or "Chit Passing". The choice of algorithms and tools was guided by the need for accuracy, robustness, and efficiency in detecting subtle anomalies in the videos. Deep learning techniques, combined with robust preprocessing and evaluation methodologies, enable the system to achieve high performance in recognizing cheating activity while minimizing false positives.

This section provides a detailed explanation of the algorithms and tools utilized, including their roles, key features, and advantages in the context of this project.

6.1 Algorithm and Tools Explanation

6.1.1 ADAM Algorithm

Adam (Adaptive Moment Estimation) is an optimization algorithm widely used in deep learning to update network weights during training. It combines the advantages of two other popular algorithms, RMSProp and Stochastic Gradient Descent (SGD) with momentum. Adam adapts the learning rate for each parameter dynamically, making it efficient and effective for training deep neural networks.

i. Stochastic Gradient Descent (SGD) :

SGD updates model weights using gradients calculated from mini-batches of data. However, it can suffer from oscillations in steep regions and slow convergence in shallow areas. Momentum improves SGD by adding a velocity term that accumulates past gradients. This approach accelerates convergence in consistent gradient directions and reduces oscillations, enhancing stability and training efficiency.

ii. Root Mean Square Propagation (RMSProp):

RMSProp adapts the learning rate for each parameter by maintaining an exponentially decaying average of squared gradients. This adjustment prevents large updates for parameters with high gradients, ensuring stability and faster convergence. RMSProp is effective for non-stationary objectives, making it well-suited for deep learning tasks.

iii. Key Features of Adam

- Adaptive Learning Rates: Adjusts the learning rate for each parameter based on the first (mean) and second (variance) moments of the gradients.
- Momentum: Incorporates the concept of momentum, which smoothens updates by considering the past gradient directions.
- Bias Correction: Corrects bias in moment estimates during early iterations.
- Computational Efficiency: Works well with large datasets and parameters without consuming excessive memory.

iv. Working of Adam

a. Initialize Parameters:

- Learning rate (α): Default is 0.001.
- Exponential decay rates for moment estimates ($\beta_1 = 0.9, \beta_2 = 0.999$).
- Small constant (ϵ) to prevent division by zero (*e.g.*, 10^{-8})

b. Update Moments:

- Compute the first moment (mean of gradients):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \cdot g_t$$

- Compute the second moment (uncentered variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

c. Bias Correction:

- Correct the bias introduced in m_t & v_t

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

d. Update Parameters:

- Adjust weights(θ_t)

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

v. Use of Adam :

In your multi-class anomaly detection project, Adam plays a crucial role during the model training phase:

- **Weight Optimization:**

Adam adjusts the weights of the convolutional and fully connected layers in the CNN model to minimize the categorical cross-entropy loss. By adapting the learning rate for each parameter, Adam ensures efficient convergence to an optimal solution.

- **Handling Data Augmentation:**

The augmented dataset introduces variability. Adam's ability to handle noisy gradients makes it particularly suited for this situation, ensuring smooth optimization despite variations.

- **Faster Convergence:**

The project trains over 50 epochs, and Adam ensures faster convergence by combining momentum and adaptive learning rates. This reduces the training time while maintaining accuracy.

- **Improved Performance:**

Anomalies, like "Using Mobile" or "Copying," require subtle feature extraction. Adam helps the CNN model learn these complex patterns effectively by maintaining stable updates to weights.

- **Generalization:**

Adam prevents overfitting by ensuring smooth weight updates, especially when combined with techniques like dropout and data augmentation.

vi. Adam Is Suitable because:

- **Adaptive Learning:** Handles dynamic behaviour in the dataset, such as variations in anomalies.
- **Robustness:** Works well with noisy gradients, which might arise due to data augmentation.
- **Efficiency:** Optimizes training with minimal manual tuning of learning rates.

6.1.2 Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a type of deep learning model designed for processing structured data like images. CNNs are particularly effective for tasks involving spatial hierarchies, such as image classification, object detection, and anomaly detection, by automatically extracting important features from images through multiple layers.

i. Key Components of a CNN

a. Convolutional Layers:

- These layers apply convolution operations to extract spatial and temporal features from the input image. Filters slide over the image to detect patterns like edges, textures, and shapes.
- Each filter produces a feature map, capturing the presence of specific features at different locations.

b. Pooling Layers:

- Pooling reduces the spatial dimensions of feature maps, retaining essential information while reducing computational load.
- Common pooling methods: Max Pooling (selects the maximum value) and Average Pooling (calculates the average).

c. Fully Connected Layers:

These layers flatten the feature maps and connect them to a dense network, enabling the model to make predictions based on the extracted features.

d. Activation Functions:

Non-linear activation functions like ReLU (Rectified Linear Unit) introduce non-linearity, enabling the network to learn complex patterns.

e. Dropout:

Dropout regularization randomly disables a fraction of neurons during training, preventing overfitting.

ii. Working of CNN

a. Feature Extraction (Convolution and Pooling Layers):

- In the anomaly detection project, the CNN takes input video frames resized to 128x128 pixels.
- Convolutional layers process the image and extract features such as:
- Edges of objects (e.g., a mobile device in "Using Mobile").

- Distinct spatial patterns indicating activities like "Peeking" or "Chit Passing."
- Pooling layers reduce the size of feature maps, ensuring computational efficiency and focusing on prominent features.

b. Classification (Fully Connected Layers):

After extracting features, the fully connected layers process this information to classify frames into one of the nine categories (e.g., "Copying," "Suspicious"). The final softmax layer outputs probabilities for each class.

c. Robustness and Regularization:

Dropout is applied during training to prevent overfitting, ensuring the model generalizes well to unseen video frames. Data augmentation helps the CNN learn from diverse scenarios, enhancing its ability to detect anomalies.

iii. Training Process of CNN in the Project

- Input Data:** Pre-processed images (resized, normalized, augmented) are fed into the CNN.
- Loss Function:** The model uses categorical cross-entropy as the loss function to measure the difference between predicted probabilities and actual labels.
- Optimizer:** The Adam optimizer is employed to adjust the weights based on the loss, ensuring efficient and accurate learning.
- Epochs and Batch Size:** The CNN is trained over 50 epochs with a batch size of 32, allowing it to iteratively refine its understanding of anomalies.

6.1.3 TensorFlow and Keras:

TensorFlow and its high-level API, Keras, play a crucial role in creating and managing the deep learning model. This model is a Convolutional Neural Network (CNN) that learns to classify activities like "chit passing," "using mobile," and "copying" by analysing images extracted from video footage. TensorFlow handles the training process, optimization, and inference stages, enabling the system to detect anomalies in real-time or from prerecorded video data.

6.1.4 OpenCV

OpenCV provides robust video and image processing capabilities. It is used to convert input videos into individual frames, resize them to the dimensions required by the model, and apply transformations like grayscale conversion to reduce complexity. This preprocessing ensures consistency in the input format, improving model accuracy and performance. By extracting relevant image data from videos, OpenCV bridges the gap between raw footage and machine learning analysis.

6.1.5 NumPy

NumPy supports efficient data manipulation throughout the project. It handles mathematical operations on pixel arrays during image normalization, a key preprocessing step that scales image pixel values between 0 and 1. This normalization enhances the performance of the CNN by standardizing input data. NumPy also helps manage image transformations and data augmentations needed for robust model training.

6.1.6 Scikit-learn

Scikit-learn enhances the evaluation phase by providing essential metrics like accuracy, precision, and confusion matrices. These metrics allow you to assess how well the model distinguishes between normal and various cheating actions. This feedback is vital for refining the model, improving its generalizability, and ensuring reliable anomaly detection.

6.1.7 Streamlit

Streamlit transforms the system into an interactive application where users can upload video files and receive predictions. It provides a user-friendly interface that displays results with intuitive emoji symbols representing detected activities. Streamlit's simplicity and dynamic visualizations make the complex process of anomaly detection accessible to non-technical users, adding practicality and ease of use to the solution. Together, these tools integrate seamlessly to create a comprehensive, intelligent monitoring system for offline exams.

6.2 Integration and Workflow

Integration: The system is designed to work smoothly by connecting its various parts—the front-end (the user interface) , back-end (video processing) and the deep learning model. The front-end, built with Streamlit, allows users to upload videos and interact with the system. Behind the scenes, the back-end processes the videos using a TensorFlow-based model trained to detect anomalies.

Workflow: When a user uploads a video through front-end, the back-end takes over to analyse it frame by frame, looking for unusual activities. The detected anomalies are highlighted and annotated on the video. Once processing is complete, the updated video is sent back to the front-end for the user to view. This streamlined workflow ensures a smooth and intuitive experience for users while delivering accurate anomaly detection results.

6.3. Code Implementation

The implementation of the system consists of three major parts: Model Training, Video processing and Anomalies Detection and User Interface (UI). These components work seamlessly to deliver a functional solution for anomaly detection in exam monitoring.

i. Dataset Preparation

Dataset Structure: Images are categorized into nine classes: Normal, Copying, Discussing, Cheat Passing, Peeking, Using Mobile, Showing Answer, Suspicious, and Using Copy Cheat. Each class represents a specific activity observed in exams.

Image Preprocessing:

- 224x224 sized frames are extracted and then resized to 128x128 for CNN training.
- Grayscale conversion is applied to reduce complexity.
- Data augmentation techniques, such as rotation, zoom, and flipping, are employed to enhance dataset diversity.

Frame Extraction from Videos: A custom script is used to extract frames from video files, ensuring a uniform dataset by resizing and storing frames as labelled images.

ii. Model Development

Model Architecture:

- A convolutional neural network (CNN) is designed with three convolutional layers, each followed by max-pooling layers to extract and down sample features.
- Fully connected layers, including a dropout layer, ensure robust predictions and prevent overfitting.
- The final softmax layer outputs probabilities for the nine classes.

Training Process:

- The model is trained on the prepared dataset using categorical cross-entropy as the loss function and the Adam optimizer for efficient learning.
- Validation split ensures model generalization, and training is conducted over 50 epochs.

iii. Video Processing and Anomaly Detection

Frame Prediction:

- Each video frame is resized and normalized before being passed through the trained CNN model.
- The model predicts the class of each frame and assigns a confidence score.

Anomaly Visualization:

- Detected anomalies are displayed on video frames using labelled text.
- Frames are processed and saved at 4K resolution to maintain clarity.

iv. User Interface

Streamlit Integration:

- The system's interface allows users to upload videos, process them, and view detected anomalies.
- A progress bar and status updates provide feedback during video processing.
- Navigation options, such as Home, Detection, and About, ensure a user-friendly experience.

Visual Enhancements:

- Background styling with gradient colors improves the application's aesthetics.
- Anomalies are highlighted with visually distinct labels and emojis for better clarity.

CHAPTER 7

TESTING AND RESULTS

7.1 Performance evaluation Metrics

The Anomaly Detection from Offline Exam Monitoring project was evaluated using a dataset with labelled categories (e.g., "copying," "normal," "discussing," "using mobile"). The model's performance was assessed through

Confusion matrix, Classification report, Model accuracy curve and model loss curve. These evaluations provided insights into the model's ability to accurately detect and classify different activities in exam scenarios.

Dataset	Training Accuracy	Validation Accuracy
Exam Dataset	87.96	84.66

The table summarizes the performance of a model trained on an "Exam Dataset" for anomaly detection. The model achieved a training accuracy of 87.96%, indicating how well it learned the patterns in the training data. The validation accuracy of 84.66% reflects its performance on unseen data, demonstrating good generalization with minimal overfitting.

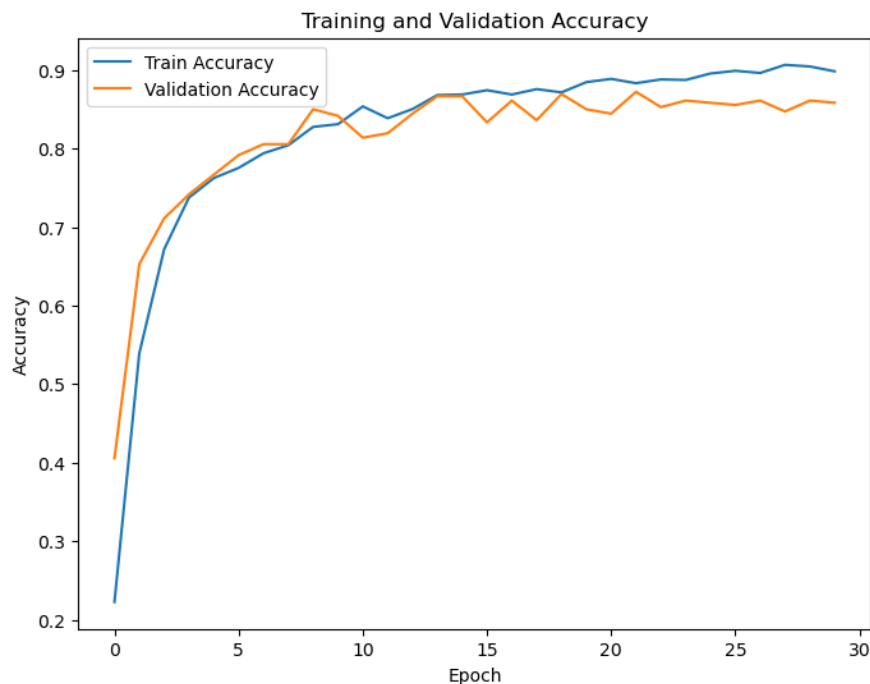


Fig 7.1 Training and Validation accuracy

This graph shows the training (blue) and validation (orange) accuracy of a model over 30 epochs. Both accuracies increase rapidly at first, with validation accuracy stabilizing after a point while training accuracy continues improving slightly.

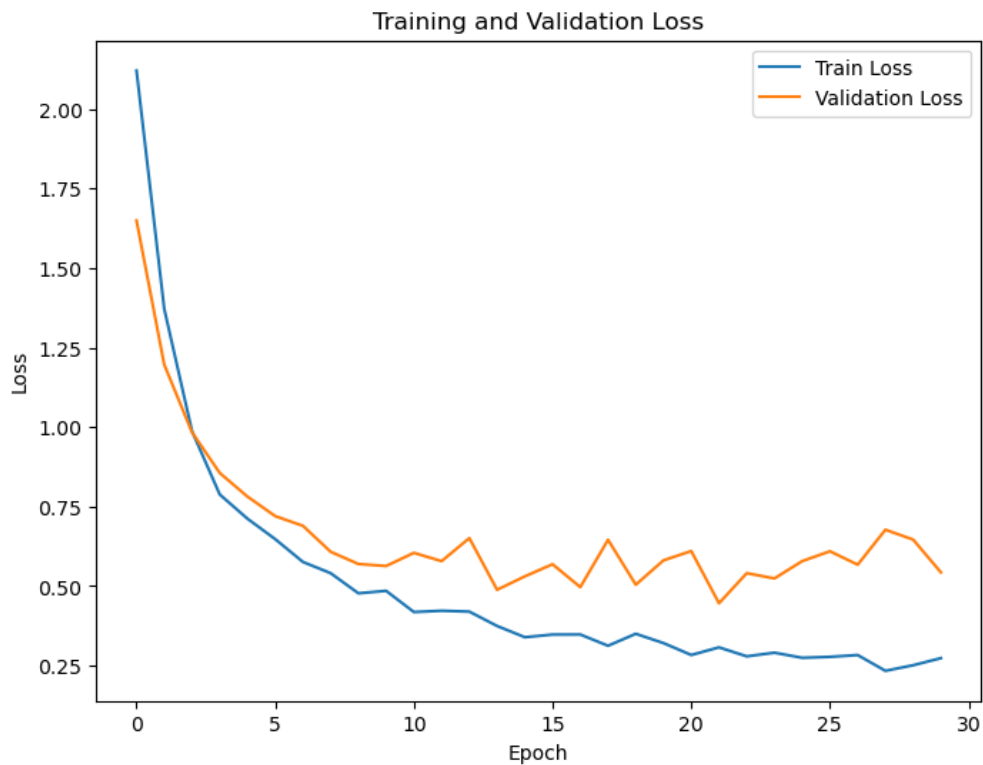


Fig 7.2 Training and Validation loss

This graph shows the training (blue) and validation (orange) loss of a model over 30 epochs. Both losses decrease rapidly at first, indicating effective learning, with training loss continuing to decline steadily.

a. Precision

Precision is the ratio of true positive predictions (correctly predicted positive instances) to the total number of positive predictions made by the model (true positives + false positives).

Formula:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Precision tells you how accurate the model's positive predictions are. A higher precision indicates fewer false positives.

b. Recall

Recall (also known as sensitivity or true positive rate) is the ratio of true positive predictions to the total number of actual positive instances (true positives + false negatives).

Formula:

$$Recall = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Recall indicates how well the model identifies positive instances. A high recall means fewer false negatives.

c. F1-Score

The F1-score is the harmonic mean of precision and recall. It provides a balance between precision and recall, especially when there is an uneven class distribution.

Formula:

$$F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is a good metric to evaluate the model when you need a balance between precision and recall, especially in cases of class imbalance.

d. Support

Support is the number of actual occurrences of the class in the dataset (i.e., the number of true instances of each class).

Support gives you an idea of how many samples belong to each class. It helps to interpret metrics such as precision and recall in the context of class distribution.

e. Accuracy

Accuracy is the ratio of correct predictions (true positives + true negatives) to the total number of predictions (all instances).

Formula:

$$Accuracy = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$

Accuracy is a general measure of how often the model makes correct predictions. However, accuracy can be misleading if the dataset is imbalanced (i.e., many instances of one class and very few of another).

	precision	recall	f1-score	support
Discussing	0.76	0.92	0.83	37
Normal	0.89	0.89	0.89	46
Peeking	0.92	0.96	0.94	57
cheat passing	0.91	0.67	0.77	45
copying	1.00	1.00	1.00	38
showing answer	0.96	0.98	0.97	49
suspicious	0.67	0.84	0.75	37
using copy cheat	0.89	0.79	0.84	42
using mobile	1.00	0.90	0.95	42
accuracy			0.89	393
macro avg	0.89	0.88	0.88	393
weighted avg	0.89	0.89	0.89	393

Fig 7.3 Classification report for anomaly detection

The classification report summarizes the performance of a model for detecting various activities or anomalies. It includes metrics such as precision, recall, and F1-score for each class (e.g., "Discussing," "Normal," "Peeking," etc.), with their respective support (number of samples). The overall accuracy is 88%, and the macro and weighted averages of precision, recall, and F1-score indicate the model's average performance across all classes.

User Interface -Home Page



Fig 7.4 Home page of user interface

This interface is for an **Anomaly Detection System for Offline Exam Monitoring**. It allows users to upload videos for automated detection of unusual activities like copying, powered by CNNs built with TensorFlow and Keras. The system features real-time frame analysis and annotation to enhance exam monitoring efficiency.

User Interface – Detection page

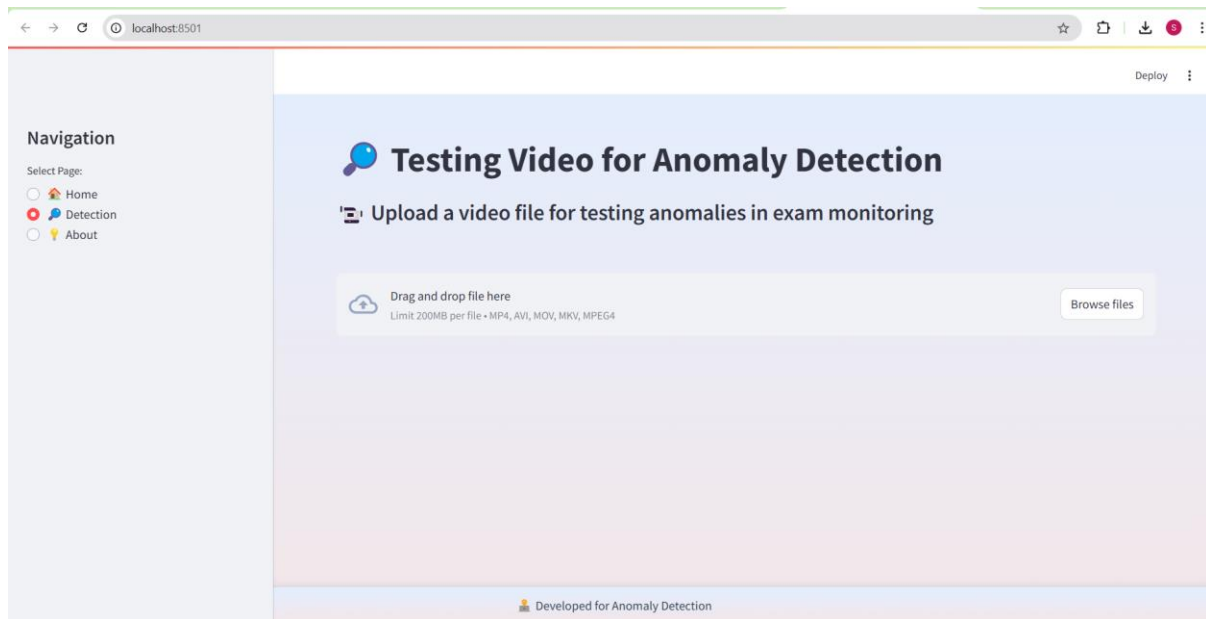


Fig 7.5 Detection page of user interface

This is a web application interface for testing video-based anomaly detection in exam monitoring. Users can upload video files (up to 200MB in formats like MP4, AVI, MOV, MKV, and MPEG4) for analysis. The navigation panel allows switching between pages like Home, Detection, and About.

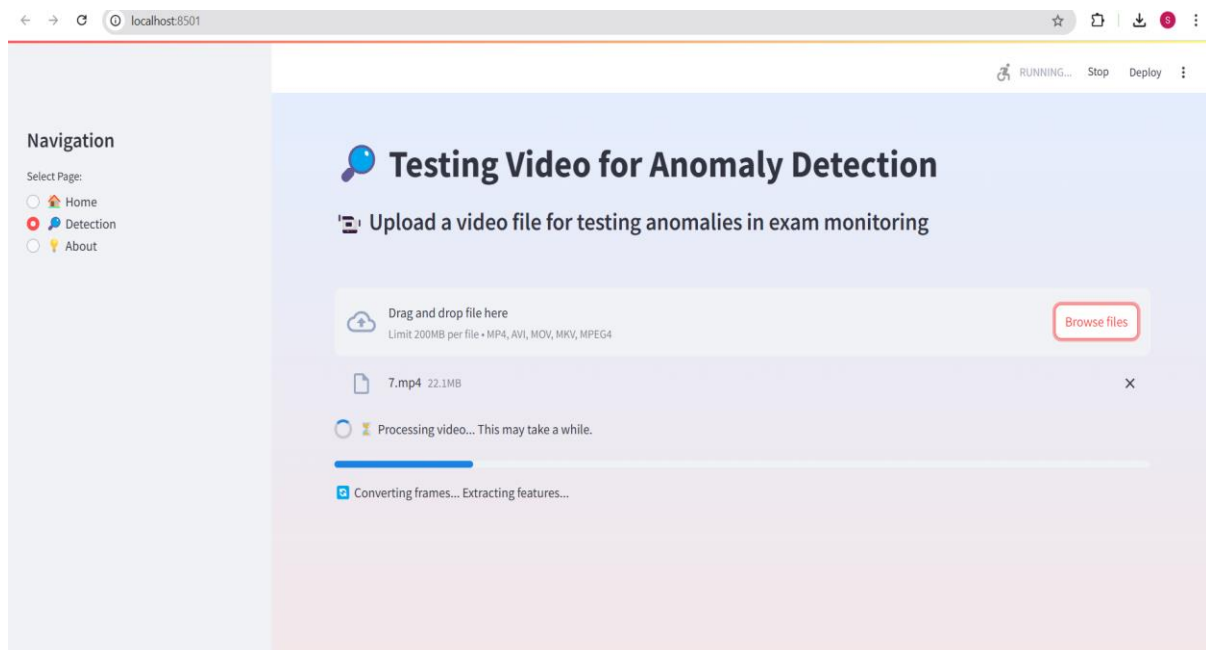


Fig 7.6 Video processing for anomaly detection

This interface shows the ongoing video processing stage in an anomaly detection system for exam monitoring. The uploaded video file ("7.mp4") is being processed, with steps like frame conversion and feature extraction and anomaly detection displayed. A progress indicator informs users that the process may take some time.

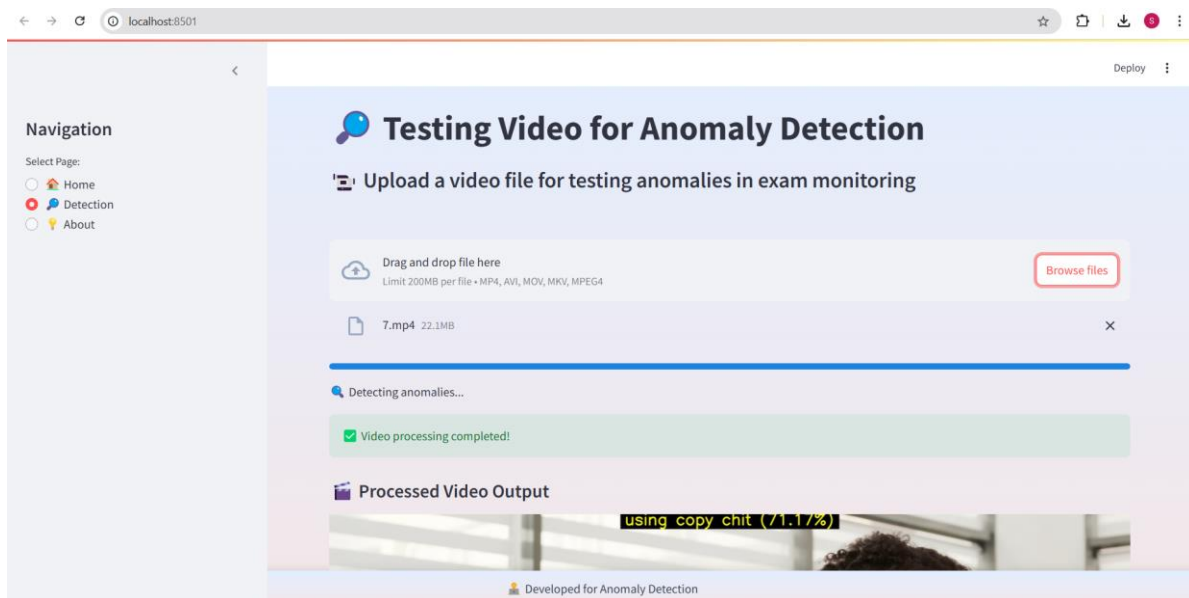


Fig 7.7 Video processing completed for anomaly detection

This interface shows the anomaly detection process in progress for an uploaded video file in an exam monitoring system. The video file (e.g., "7.mp4") is being processed, and anomalies are detected with results displayed below. A processed video output highlights detected anomalies visually for review.

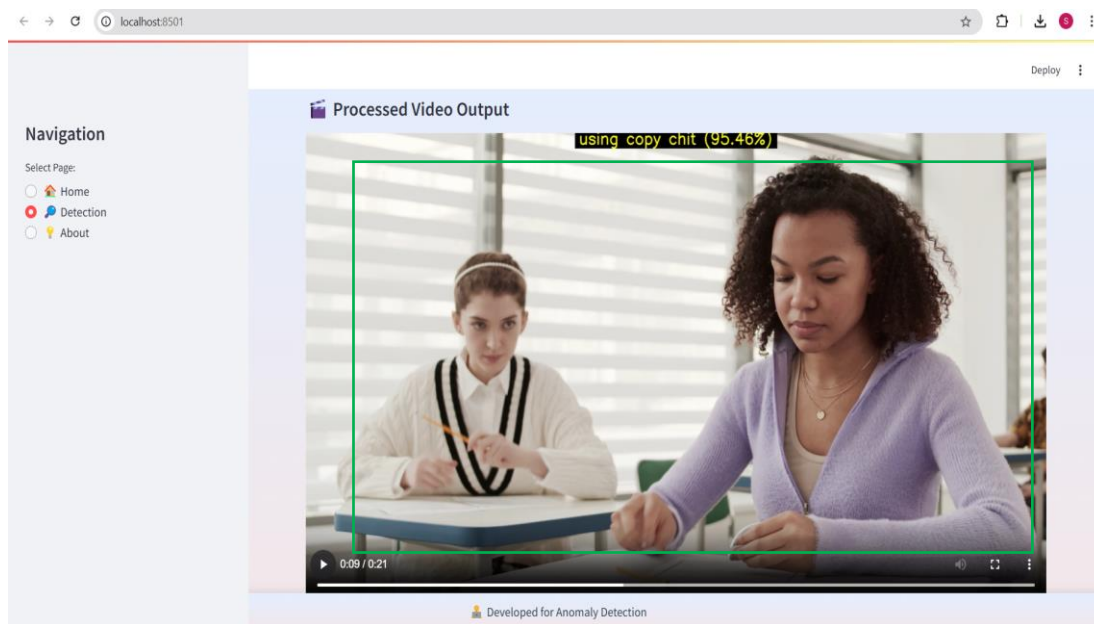


Fig 7.8 Processed video of anomaly detection

This interface shows the anomaly detection process where a uploaded video is being analyzed with a label, "using copy chit (95.46%)," indicating possible cheating detection.

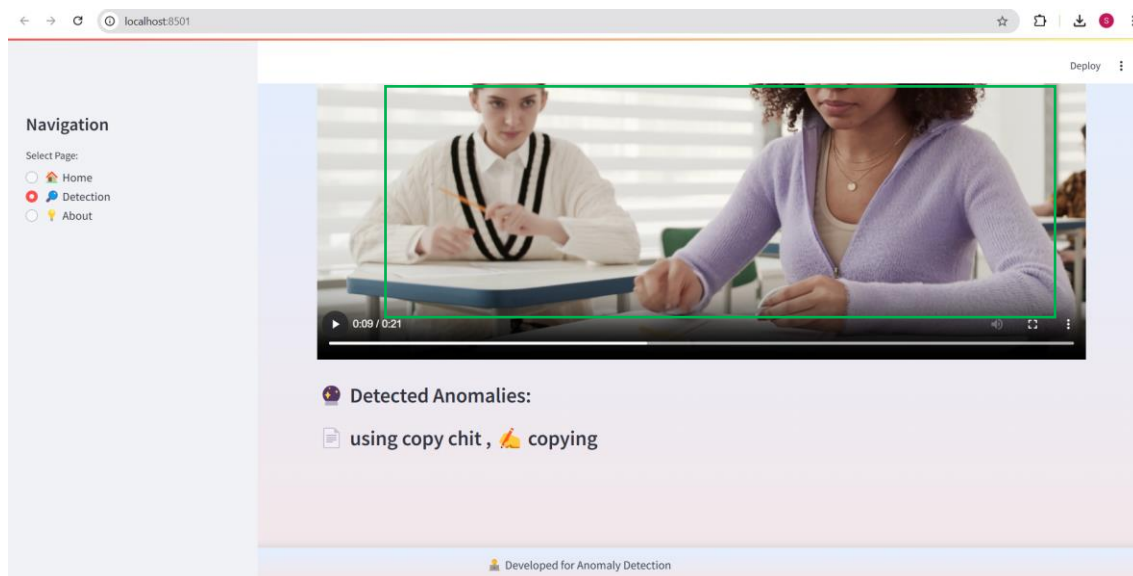


Fig 7.9 Detected anomalies are listed

This web application interface for offline exam monitoring highlights detected anomalies like "using copy chit" and "copying" in an analysed video.

User Interface – About page

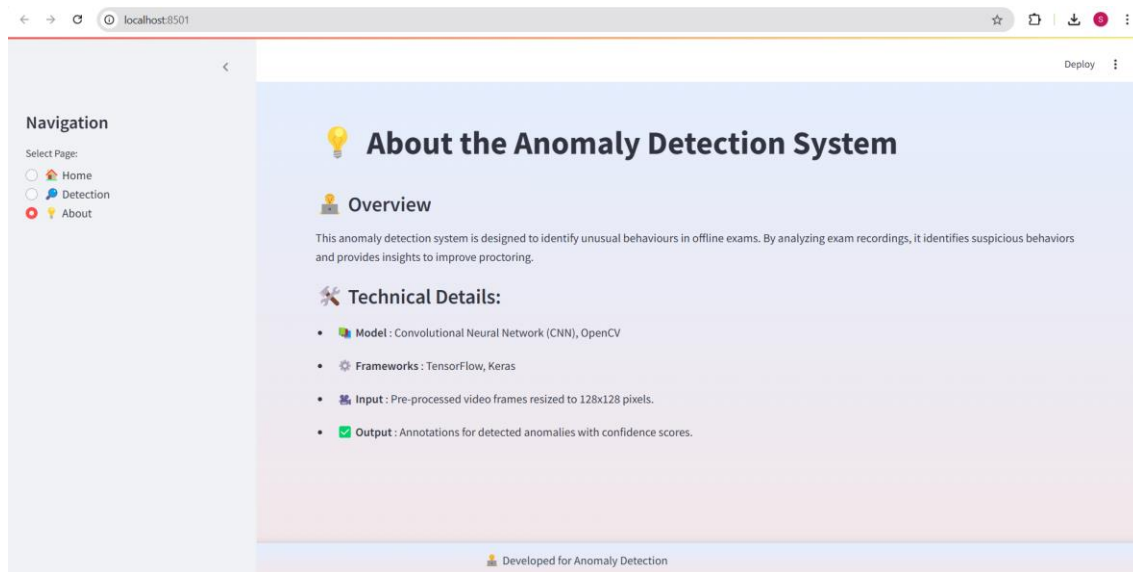


Fig 7.10 About page of user interface

The image shows the "About the Anomaly Detection System" page of a web application. It provides an overview of the system, designed to detect unusual activities in offline exams using CNNs, OpenCV, TensorFlow, and Keras. The technical details highlight input as resized video frames and output as annotated anomalies with confidence scores.

CONCLUSION

The current invigilation methods often fail to detect subtle cheating activity during the examination, especially in large-scale examination setups. This project addresses the growing need for identifying malpractices using computerized solution using advanced Convolutional Neural Networks (CNN) used for object detection for suspicious activities.

we designed a deep learning-based system that leverages Convolutional Neural Networks (CNNs) to detect and classify suspicious activities. The model was trained on diverse datasets containing various cheating scenarios, such as cheat-passing, copying, and using mobile phones. By combining advanced machine learning techniques with an intuitive interface, the system effectively identifies and highlights unusual activities and achieving a significant success rate in identifying these anomalies.

The front-end, built with Streamlit, ensures a smooth user experience by allowing video uploads and presenting results in a visually appealing format. Each analysed video frame is annotated with detected activities, confidence scores, offering clear insights into suspicious activities.

The proposed system demonstrated a success rate of 88% in detecting anomalies, making it a useful tool for detecting anomalies in exam hall during offline examination.

REFERENCES

- [1] Neha Soman, Renuka Devi M.N, Gowri Srinivasa, “Detection of Anomalous Behavior In an Examination Hall Towards Automated Proctoring” IEEE Conference Publication 2017.
- [2] Shubhangi D C, Mohammed Abdul Waheed et al , “Deep Neural Network Based Anomaly Detection For Smart Surveillance System” , IEEE North Karnataka Subsection Flagship International Conference (NKCon) 2022.
- [3] Maryam Qasim , Elena Verdu, “ Video anomaly detection system using recurrent and deep convolutional models” , Results in Engineering 2023.
- [4] Manal Mostafa Ali, “Real-time video anomaly detection for smart surveillance”, IET Image Process 2023.
- [5] Rashmiranjan Nayak, Umesh Chandra Pati, Santos Kumar Das, “ A comprehensive review on deep learning-based methods for video anomaly detection ”, Image and Vision Computing 2021.
- [6] Huu-Thanh Duong ,Viet-Tuan Le and Vinh Truong Hoang, “Deep Learning-Based Anomaly Detection in Video Surveillance”, Sensors 2023.
- [7] Ahmed Elmetwally, Reem Eldeeb, Samir Elmougy , “Deep learning based anomaly detection in real-time video”, Multimedia Tools and Applications 2024.
- [8] Akmalazakia Fatan Derana Marsiano, Indah Soesanti, Igi Ardiyanto(2019). “Deep learning-based Anomaly Detection on Surveillance Videos: Recent Advances”, IEEE Conference Publication 2019.
- [9] Amira Ben Mabrouk , Ezzeddine Zagrouba, “Abnormal behavior recognition for intelligent video surveillance systems”, Expert Systems With Applications 2017.